

SDS — Exam-Time Cheat Sheet

Scalable Data Science • LSH • Streaming • PageRank • Communities • Spark • Answer Pattern

1. SIMILARITY SEARCH & LSH

Jaccard

$J(A,B) = |A \cap B| / |A \cup B| \quad d_j = 1 - J$

Use for **set / binary membership** data: users–songs, docs–words, machines–IPs, patients–symptoms. Ignores counts.

MinHash — key property

$P[h(A) = h(B)] = J(A,B)$

Each hash = one permutation; signature = vector of m mins. Estimate ■ = (# matching positions) / m.

LSH banding

$P(\text{candidate}) = 1 - (1 - s^r)^b$

Signature length $m = b \times r$. S-curve threshold $\approx (1/b)^{1/r}$. More **r** = stricter; more **b** = looser.

(b,r)	m	P(s=.8)	P(s=.5)
b=10, r=7	70	0.905	0.075
b=20, r=5	100	0.999	0.273
b=5, r=10	50	0.610	0.005

Other LSH families

- **Cosine** → random hyperplane: $P[\text{match}] = 1 - \theta/\pi$
- **Euclidean** → random projection onto line, bucket by width w
- **Shingles + MinHash + LSH** = near-duplicate documents

Common traps

- LSH returns **candidates**, not nearest neighbors — verify with exact J.
- Confusing similarity vs distance (J vs d.).
- Treating play counts in plain Jaccard (it's set-based).

2. STREAMING / SKETCHES

If question asks...	Use
"Have we seen this item?"	Bloom filter
"How often did X appear?"	Count-Min Sketch
"How many distinct items?"	HyperLogLog
"Random sample, unknown N"	Reservoir sampling
"Stable fraction of stream"	Hash-based deterministic sampling
"Count 1's in last N bits"	DGIM
"2nd moment of stream"	AMS sketch

Bloom filter

$m \approx -n \cdot \ln(p) / (\ln 2)^2 \quad k \approx (m/n) \cdot \ln 2$

- False positives possible, **no false negatives** (no deletions).
- Test only — no count, no listing of items.

Count-Min Sketch

$d \times w$ table; est = min over d rows

- **Always $\geq$ true count** (overestimates).
- $w = e/\epsilon$, $d \approx \ln(1/\delta)$.
- Conservative update: bump only the row(s) at the min.

HyperLogLog

Hash items, track max trailing-zero run per bucket. Estimate distinct count from harmonic mean of 2^{max} . Tiny memory (~few KB) for billions of items.

Reservoir sampling (size k)

Keep first k. For $i > k$, accept item i with prob k/i ; if accepted, replace random reservoir slot. Final $P(\text{item in sample}) = k/N$.

DGIM (1's in last N)

Buckets of powers-of-2 1's, ≤ 2 of each size. Memory $O(\log^2 N)$. Error $\leq 50\%$ of last bucket size.

Traps: Bloom $\neq$ counter. CMS doesn't underestimate. Reservoir doesn't need N. HLL doesn't do membership.

3. LINK ANALYSIS

PageRank (one score)

$PR(v) = (1-d)/N + d \cdot \sum_{u \in \text{In}(v)} PR(u)/\text{OutDeg}(u)$

- Damping $d \approx 0.85$; teleport $(1-d)$ prevents sinks/spider traps.
- Iterate to convergence ($||\Delta PR|| < \epsilon$).
- **Dangling nodes**: redistribute their mass uniformly each iter.

Personalized / Topic-Sensitive PR

teleport → vector **e** biased to seed set **S**

Use when ranking **relative to a target group** (e.g., residential nodes, a topic, a user's history).

HITS — two scores

$a(v) = \sum_{u \rightarrow v} h(u) \quad h(u) = \sum_{u \rightarrow v} a(v)$

Normalize each iteration. **Hubs** point to many good authorities; **authorities** are pointed to by many good hubs. Computed on a query-induced subgraph (root + base set).

When to use which

If question asks...	Use
"Most important node overall"	PageRank
"Important relative to group X"	Personalized PR
"Hub vs authority roles"	HITS
"Recommend via random walk"	Personalized PR / SimRank

Power method needs **row-stochastic** matrix. Treat dangling nodes; don't forget normalization in HITS.

SDS — Exam-Time Cheat Sheet · p. 2

Communities · Triangles · Spark patterns · Answer template

4. COMMUNITY DETECTION

Modularity

Q = (1/2m) Σ_{ij} [A_{ij} - k_ik_j/(2m)] · δ(c_i, c_j)

Compares observed within-community edges vs random expectation. Q ∈ [-0.5, 1]; Q > 0.3 typically indicates real community structure. Resolution limit: small communities can be missed in big graphs.

Girvan-Newman

1) Compute edge betweenness; 2) remove highest; 3) recompute; 4) repeat → dendrogram. Cut at the level with highest Q. Costly: O(VE) per iteration (Brandes).

BC(e) = Σ_{s≠t} σ_{st}(e) / σ_{st}

Louvain (modularity-based)

Greedy: each node joins neighbor's community if ΔQ > 0; collapse and repeat. Fast, scalable, popular default.

Other methods

If question asks...	Use
Detect by removing edges	Girvan-Newman
Optimize modularity at scale	Louvain
Fast, no objective needed	Label propagation
Overlapping communities	Clique Percolation (CPM)
Cut-based, balanced parts	Spectral / METIS

SimRank (structural similarity)

s(a,b) = C / (|I(a)|·|I(b)|) · Σ_{i∈I(a)} Σ_{j∈I(b)} s(i,j)

"Two nodes are similar if their in-neighbors are similar." Recursive; expensive (O(N²) memory). C ≈ 0.6–0.8 decay.

5. TRIANGLES & CLUSTERING

Global clustering coefficient

C = 3 · #triangles / #connected-triples

Factor of 3 because each triangle contains 3 connected triples.

Local clustering coefficient

C_i = 2·E_i / (k_i(k_i-1))

E_i = edges among i's neighbors. Measures triadic closure.

Triangle counting (scalable)

- Naive: O(V³) — don't.
- **Nodeiterator+**: for each node, intersect neighbor lists.
- **Order by degree** (low → high) so each triangle counted once → O(E·√E) for sparse graphs.
- **Spark**: self-join edges, filter u<v<w.

6. SPARK PATTERNS

from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("x").getOrCreate()
df = spark.read.csv(path, header=True, inferSchema=True)

Build user-item sets (LSH input)

sets = (df.dropna().dropDuplicates(["uid", "iid"])
 .groupBy("uid")
 .agg(F.collect_set("iid").alias("items")))

Build directed edges (graph input)

edges = df.select(F.col("src").alias("src"),
 F.col("dst").alias("dst")).distinct()

Iterative PageRank skeleton

ranks = nodes.withColumn("r", F.lit(1.0/N))
for _ in range(K):
 contribs = ranks.join(edges, "src")
 .withColumn("c", F.col("r")/F.col("out"))
 ranks = (contribs.groupBy("dst")
 .agg(F.sum("c").alias("r"))
 .withColumn("r", 0.15/N + 0.85*F.col("r")))
 ranks.cache()

- Use **aliases** on join keys (avoid ambiguous-column errors).
- **cache()** across iterations; **checkpoint** every ~10 to truncate lineage.
- Avoid **collect()** on big DFs. Restart kernel after JVM crash.

7. THE D-M-F-I-S-I-L ANSWER PATTERN

Every SDS question gets the same skeleton. Don't skip steps even if brief.

Step	What to write
D	Data representation (sets? graph? stream?)
M	Method choice + why it fits
F	Formula / algorithm
I	Implementation plan (Spark sketch)
S	Scalability / complexity argument
I	Interpretation in domain language
L	Limitations / failure modes

Master decision table

If question asks...	Use
Set overlap at scale	MinHash + LSH
Near-duplicate docs	Shingle + MinHash + LSH
Vector cosine at scale	Random hyperplane LSH
Membership, low memory	Bloom filter
Stream frequency	Count-Min Sketch
Distinct count	HyperLogLog
Random sample, stream	Reservoir
Counts in sliding window	DGIM
Node importance (global)	PageRank
Node importance vs group	Personalized PageRank
Hubs vs authorities	HITS
Find communities	Louvain (or Girvan-Newman small)
Overlapping communities	Clique percolation
Structural similarity	SimRank
Triadic closure	Clustering coefficient

Strong-answer template (LSH example)

"I represent each user as a set of song IDs and use Jaccard similarity because the task is set overlap. All-pairs Jaccard is O(n²), so I use MinHash–LSH: m=70 hashes, b=10, r=7, giving P(s=.8)≈.91 and P(s=.5)≈.08. I generate candidates by band collisions in Spark, then verify with exact Jaccard. Limitation: ignores play counts; tunable via b,r."

Final reminders

- Always state representation explicitly.
- Plug numbers into formulas — show the arithmetic.
- Name the scalability win (sublinear memory? candidate pruning? parallel partitioning?).
- Close with one limitation — earns interpretation points.